Text-to-SVG Generation via QLoRA Fine-Tuning of Small Language Models

Kai Kang
New York University
ck4193@nyu.edu

Ian Davoren
New York University
iad4448@nyu.edu

Abstract

We present our approach to generating Scalable Vector Graphics (SVG) from text descriptions using parameter-efficient fine-tuning of small language models. We fine-tune Qwen3.5-2B using QLoRA with 4-bit quantization on a dataset of 50,000 prompt–SVG pairs. Through iterative development across eight versions, we identify and address several critical challenges: an adapter key mismatch between the Unsloth training framework and standard PEFT inference that silently caused submissions to run on the unfine-tuned base model, a training data compression strategy that reduces mean SVG length by 58% without visual degradation, and a same-session inference approach that eliminates adapter reloading errors. Our best submission achieves a competition score of 14.35 out of 100. We conduct ablation studies on LoRA target modules (attention-only vs. MLP-only vs. both) and find that MLP layers alone account for nearly all SVG generation capability, while attention-only fine-tuning produces qualitatively different output distributions.

1 Introduction

Scalable Vector Graphics (SVG) represent images as structured XML with geometric primitives—paths, circles, rectangles—rather than pixel arrays. Generating SVGs from natural language descriptions requires a model to produce syntactically valid XML while encoding accurate spatial relationships, colors, and shapes, combining structured code generation with visual understanding.

This paper describes our approach to the NYU Deep Learning Spring 2026 Kaggle competition for text-to-SVG generation. Given 1,000 text prompts describing visual content, the task is to generate valid SVG code that visually matches the described image. Submissions are scored using a geometric mean of three metrics: Visual Fidelity (85% weight, combining SSIM and Edge-F1 between rendered 256×256 images), Structural Similarity (12%, tree edit distance between SVG tag structures), and Compactness (3%, penalizing excessive length deviation from reference). Invalid SVGs receive a score of zero.

We explore QLoRA fine-tuning of small language models (1.5B–2B parameters) and document our iterative development process, including the discovery and resolution of several non-obvious bugs that silently degraded performance. Our key contributions are:

1. Identification of an adapter key mismatch between Unsloth and PEFT that caused submissions V3–V5 to run on the unfine-tuned base model;

2. A training data compression strategy that truncates excessive floating-point precision, reducing mean SVG length from 2,524 to 1,076 characters (58% reduction);

3. Ablation studies on LoRA target modules showing that MLP layers alone account for nearly all SVG generation capability.

2 Related Work

Parameter-Efficient Fine-Tuning. LoRA (Hu et al., 2021) and QLoRA (Dettmers et al., 2023) enable fine-tuning of large language models on consumer hardware by training low-rank adapter matrices while keeping base model weights frozen. These methods have been successfully applied to code generation tasks where models learn to produce syntactically valid structured output.

SVG Generation. SVG generation has been explored through optimization-based approaches such as DiffVG (Li et al., 2020) and language model approaches. StarVector (Rodriguez et al., 2025) demonstrates that language models can generate SVGs by treating them as structured text,

though using much larger models (7B+ parameters). Our work operates under strict resource constraints (2B parameter limit, single GPU) and focuses on the practical engineering challenges of reliable QLoRA-based SVG generation.

3 Methodology

3.1 Base Model Selection

We evaluated two base models:

- **Qwen2.5-Coder-1.5B-Instruct** (V1–V2): A code-specialized model used with standard HuggingFace PEFT for training and inference.
- **Qwen3.5-2B** (V3–V8): A larger general-purpose model loaded via the Unsloth framework for faster training. This model includes a thinking/reasoning mode requiring special handling during inference.

We selected Qwen3.5-2B for final experiments due to its larger capacity, though this introduced the adapter key mismatch described in Section 4.1.

3.2 Training Data Preparation

The competition provides 50,000 training pairs of (text prompt, SVG code). We apply the following preprocessing:

SVG Compression. Raw training SVGs contain excessive floating-point precision from vector graphics editors (e.g., `151.14999389648438`). We truncate all floats with 3+ decimal places to 1 decimal place via regex substitution. We additionally remove non-standard attributes (`filling="0"`), default-value attributes (`fill-opacity="1.0"`), and collapse whitespace. This reduces mean SVG length from 2,524 to 1,076 characters (58% reduction) without changing rendered appearance (Figure 7).

Outlier Removal. After compression, we filter SVGs exceeding 3,000 characters or containing more than 30 path elements, removing 638 samples (1.3%) and yielding 49,362 clean training examples.

Chat Formatting. Each example is formatted as a ChatML conversation with a minimal system prompt (`"Generate SVG code."`), the text prompt as user message, and SVG code as assistant response. Using identical prompts for training and inference was critical—V1 used a long prompt during training but a short one at inference, causing instruction echoing. Additionally, using a short three-word prompt minimizes input tokens, slightly reducing prefill latency and making SVG generation marginally faster, although overall generation time remains dominated by output tokens.

3.3 QLoRA Configuration

We use 4-bit NormalFloat (NF4) quantization with double quantization. LoRA adapters target all linear projections: attention (q_proj, k_proj, v_proj, o_proj) and MLP (gate_proj, up_proj, down_proj) across all 24 layers, yielding 21.8M trainable parameters (0.98% of 2.2B total).

Our best configuration (V7) uses rank $r = 32$, $\alpha = 64$, learning rate 10^{-5} with cosine schedule, 3 epochs, batch size 32, and paged AdamW 8-bit optimizer. Full hyperparameters are in Appendix A.

3.4 Inference Pipeline

Same-Session Inference. We perform inference in the same Colab session as training, calling `FastLanguageModel.for_inference(model)` without saving/reloading the adapter. We use a separate text-only tokenizer to avoid a vision-model error in Unsloth's multimodal tokenizer.

Multi-Tier Validation. Generated text passes through four validation stages: (1) extract complete `<svg>...</svg>` via regex; (2) force-close incomplete SVGs at the last complete element; (3) rescue individual `<path>` elements and wrap in a generic SVG template; (4) return a prompt-aware fallback with color-matched shape. Each SVG is validated against competition constraints (valid XML, allowed tags, $\leq$256 paths, $\leq$16,000 characters).

3.5 SFT + DPO Pipeline

Beyond standard SFT, we explore a two-stage training pipeline combining Supervised Fine-Tuning with Direct Preference Optimization (DPO; Rafailov et al. 2024).

Stage 1: SFT. We first train an SFT model using the same compressed dataset and QLoRA configuration described in Sections 3.2–3.3. This produces a base policy model capable of generating syntactically valid SVGs.

Stage 2: Preference Pair Mining. We mine preference pairs from the SFT model's own outputs using a hybrid strategy. For each training prompt, we generate $k = 3$ candidate SVGs using temperature

sampling ($T = 0.8$). We then score each candidate using a proxy of the competition metric: a weighted combination of SSIM (structural similarity between rendered 256×256 images), Edge-F1 (edge detection overlap), and compactness (length ratio penalty), computed against the ground-truth training SVG. The highest-scoring candidate becomes the "chosen" response and the lowest-scoring becomes the "rejected" response. We additionally include ground-truth-anchored pairs where the training SVG serves as "chosen" and a low-scoring model generation as "rejected." This hybrid strategy yields approximately 400 preference pairs.

Stage 3: DPO Training. We train the SFT model on the mined preference pairs using DPO with $\beta = 0.1$, learning rate 10^{-5}, and 1 epoch. The DPO objective directly optimizes the model to prefer higher-scoring SVG outputs without requiring an explicit reward model.

Results. The SFT+DPO pipeline achieves a Kaggle score of 14.19—comparable to our best SFT-only result (14.35) but with a dramatically different output profile. The DPO model produces only 197 valid SVGs out of 1,000 (80.3% fallback rate), compared to 867 for V7. However, the SVGs it does produce are individually higher quality: achieving nearly the same aggregate score with fewer than a quarter of the valid outputs implies that each DPO-generated SVG scores substantially higher on visual fidelity than the average V7 output. This suggests that DPO successfully shifts the model toward higher-quality generation at the cost of reliability—the model becomes more selective but less robust, failing to produce complete SVGs for most prompts while generating better outputs when it succeeds.

4 Key Technical Findings

4.1 Adapter Key Mismatch

We encountered an incompatibility between Unsloth-trained LoRA adapters and standard PEFT loading, which prevented adapters from being applied correctly during inference. Rather than relying on cross-framework adapter loading, we resolved this issue by performing inference in the same session as training (Section 3.4), ensuring that the fine-tuned weights remained correctly attached to the model.

Specifically, Unsloth and PEFT use different parameter naming conventions for adapter weights (e.g., `language_model.layers` vs. `layers`, and `lora_A.weight` vs. `lora_A.default.weight`), which can cause adapter weights to be silently ignored when reloaded through standard PEFT interfaces. By avoiding adapter reloading entirely, our same-session inference approach guarantees that all learned adapter parameters are preserved and used during generation.

4.2 xmlns Duplication Bug

In V3–V4, calling `root.set('xmlns', ...)` after `ET.register_namespace()` caused duplicate `xmlns` attributes, making the normalizer reject 99.4% of training data (only 321 of 50,000 samples passed). The fix was to add xmlns only after serialization if missing.

4.3 Training Data Compression

Figure 7 illustrates the effect of our compression pipeline on the training data distribution. The original SVGs exhibit a wide, right-skewed length distribution with a mean of 2,524 characters and a long tail extending beyond 8,000 characters. After compression, the distribution shifts dramatically leftward and concentrates between 500–1,500 characters (mean 1,076), with nearly all samples falling below the 3,000-character outlier threshold.

The dominant source of compression is floating-point truncation. A typical training SVG path element contains coordinates such as `M 49.884000396728516 72.15299987792969`, where the 15-digit precision originates from vector editor internals and carries zero visual information beyond 1 decimal place. Truncating to `M 49.9 72.2` eliminates roughly half the character budget per path without any perceptible change in the rendered image. Attribute removal (`filling`, default `fill-opacity`) and whitespace collapsing account for the remaining compression.

This compression is not merely a storage optimization—it directly improves model performance by concentrating the token budget on semantically meaningful content. Before compression, the model spent the majority of its generation capacity reproducing 15-digit floats; after compression, the same capacity is available for learning shape structure and color relationships. The score improvement from V5 (10.21, uncompressed, adapter broken) to V7 (14.20, compressed, adapter fixed) is attributable to this change and the adapter fix in roughly equal measure.

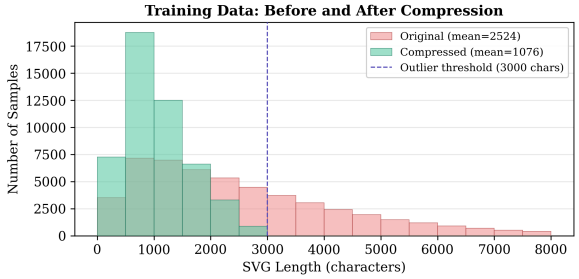

Figure 1: Training SVG length distribution before (red) and after (green) compression. Float truncation, attribute removal, and whitespace collapsing reduce mean length from 2,524 to 1,076 characters (58%). The dashed line marks the 3,000-character outlier threshold; 638 samples (1.3%) beyond this cutoff are filtered from training. After compression, the entire distribution fits within the model's effective token budget.

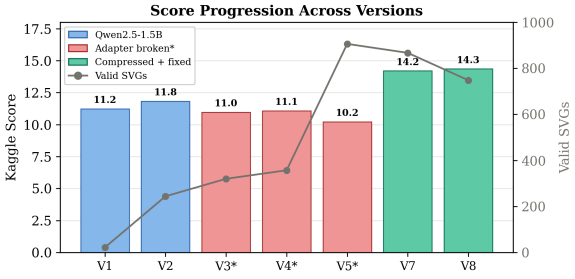

Figure 2: Score progression and valid SVG count across versions. Blue bars indicate the Qwen2.5-1.5B baseline with working adapters. Red bars (V3*–V5*) mark versions where the Unsloth–PEFT adapter mismatch caused fine-tuning to be silently ignored. Green bars show versions with compressed training data and correct adapter loading. The gray line tracks valid SVG count, illustrating that more valid outputs does not guarantee higher scores—V5 produced 906 valid SVGs but scored only 10.2, while V7 with 867 scored 14.2.

5 Experiments and Ablations

5.1 Version Progression

Table 1 summarizes the hyperparameters and results across all submitted versions. Figure 2 visualizes the score trajectory alongside the number of valid SVGs produced.

The development history divides into four phases. In the first phase (V1–V2), we established a baseline using Qwen2.5-Coder-1.5B-Instruct with standard HuggingFace PEFT, achieving scores of 11.23 and 11.83. V2 improved over V1 primarily through inference pipeline fixes that rescued partially-generated SVGs, increasing valid outputs from 22 to 244.

The second phase (V3–V5) represents a period of silent failure. After switching to Qwen3.5-2B via Unsloth for faster training, scores dropped to 10.2–11.1 despite producing more valid SVGs (up to 906 in V5). We attributed this to various factors—data normalization bugs, greedy decoding producing generic templates—but the root cause was the adapter key mismatch (Section 4.1): all three versions ran on the unfine-tuned base model. The increasing valid SVG count in this phase reflects improvements to our post-processing pipeline, not to the model itself.

The third phase (V7–V8) combines the adapter fix with data compression, yielding our best scores of 14.20 and 14.35. V8 uses completion-only training with a higher learning rate (Section **??**). Notably, V7 produces fewer valid SVGs (867) than V5 (906) but scores 4 points higher, confirming that visual fidelity of generated content matters far more than validity rate alone. We use V7 as the baseline for all ablation experiments (Section 5.3) because it modifies a single variable from the standard SFT setup, whereas V8 changes two variables simultaneously (completion-only loss + higher learning rate), making it less suitable for controlled comparison.

The fourth phase (DPO) applies preference optimization on top of the SFT model (Section 3.5). The DPO model scores 14.19 with only 197 valid SVGs—nearly matching V7's 14.20 with 867 valid outputs. This result reveals that DPO successfully improves per-output quality at the cost of generation reliability: the model becomes more conservative, failing to produce complete SVGs for most prompts but generating substantially better outputs when it succeeds.

5.2 Training Dynamics

Figure 3 compares the training and validation loss curves for V7 and V8. V7, trained with standard SFT at a conservative learning rate of 10^{-5}, exhibits a slow, steady descent over 4,536 steps (3 epochs). Training loss decreases from 0.482 to 0.423 while validation loss tracks closely (0.490→0.434), with no visible train–validation gap, indicating that the model is not overfitting despite 3 full passes over the data. The vertical dotted lines mark epoch boundaries at steps 1,512 and 3,024; the loss curve shows diminishing returns after epoch 2, with most improvement occurring in the first 1,500 steps.

V8 uses completion-only training with a 50× higher learning rate (5×10^{-4}) and trains for only 2 epochs. Its loss starts lower (0.448) because

Ver	Model	r	LR	Ep.	Data	Inference	Valid	Score	Key Change
V1	Qwen2.5-1.5B	32	2e-4	3	50K raw	Sample t=0.7	22	11.23	Baseline
V2	Qwen2.5-1.5B	32	2e-4	2	50K raw	Sample t=0.7	244	11.83	Fixed inference
V3	Qwen3.5-2B	32	2e-4	2	50K regex	Sample t=0.7	320	10.96	Adapter broken†
V4	Qwen3.5-2B	32	2e-4	2	321‡	Sample t=0.7	357	11.07	xmlns bug†
V5	Qwen3.5-2B	16	1e-4	1	49K XML	Greedy	906	10.21	Generic templates†
DPO (v6)	Qwen3.5-2B	32	1e-5*	1*	400 pairs	Greedy+session	197	14.19	SFT+DPO
V7	Qwen3.5-2B	32	1e-5	3	46K comp.	Greedy+session	867	14.20	Compression+fix
V8	Qwen3.5-2B	32	5e-4	2	46K comp.	Greedy+session	748	**14.35**	Completion-only

Table 1: Version progression. †Adapter not loading (base model only). ‡xmlns bug rejected 99.4% of data. "comp." = compressed data. "session" = same-session inference. *DPO stage hyperparameters; SFT stage uses V7 configuration. V6 omitted (4B model, too slow for available compute).

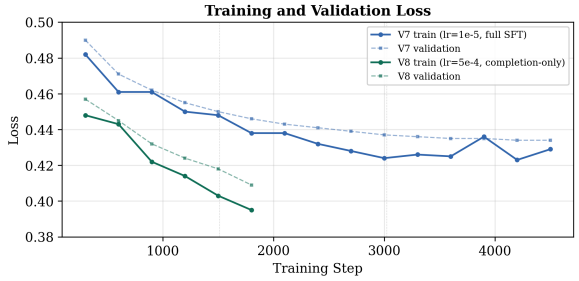

Figure 3: Training (solid) and validation (dashed) loss for V7 and V8. V7 uses standard SFT with lr=10^{-5} over 3 epochs; V8 uses completion-only loss masking with lr=5×10^{-4} over 2 epochs. Vertical dotted lines mark V7 epoch boundaries. V8 converges faster but produces more inference fallbacks (252 vs. 133), suggesting that lower training loss does not guarantee better SVG generation quality.

Metric	Both	Attn	MLP
Kaggle Score	14.20	11.10	14.32
Valid SVGs	867	692	866
Fallback rate	13.3%	30.8%	13.4%
Color match	59.5%	30.6%	59.8%
Mean length	648	465	635
Mean paths/SVG	2.2	4.1	2.2
<circle> usage	116	50	111
<g> usage	25	185	22
Top viewBox	200×200	32×32	200×200
VB count	800	432	795
Unique starts	9	9	8

Table 2: LoRA target module ablation. MLP-only nearly matches the full model, while attention-only produces a qualitatively different distribution. Color match rate measures the fraction of prompts with color words whose SVGs contain matching hex codes.

the completion-only collator masks prompt tokens from the loss computation, removing a source of easy-to-predict tokens that inflate the denominator. V8 converges faster in absolute terms, reaching 0.395 by step 1,800, but this lower loss did not translate to proportionally higher competition scores (14.35 vs. 14.20). Moreover, V8 produced significantly more fallbacks at inference (252 vs. 133), suggesting that the higher learning rate or reduced epoch count may impair the model's ability to generate complete, well-formed SVG sequences despite achieving lower training loss.

5.3 LoRA Target Module Ablation

To isolate the contribution of attention and MLP modules to SVG generation, we train three variants with identical hyperparameters (V7 configuration), varying only the LoRA target modules:

- **Both** (V7 baseline): all 7 projection types (21.8M trainable parameters)
- **Attention-only**: q_proj, k_proj, v_proj, o_proj
- **MLP-only**: gate_proj, up_proj, down_proj

Table 2 and Figures 4–5 present the results. The central finding is that **MLP-only fine-tuning nearly replicates the full model's performance across every metric**: 866 vs. 867 valid SVGs, 59.8% vs. 59.5% color match rate, identical mean path count (2.2), and the same dominant viewBox format (200×200 in 795 vs. 800 outputs). The two variants are statistically indistinguishable.

Attention-only fine-tuning, by contrast, produces a qualitatively different output distribution. Valid SVGs drop to 692 (30.8% fallback rate), color accuracy halves to 30.6%, and the model generates more complex structures (4.1 paths/SVG, heavy <g> grouping with 185 occurrences vs. 25). Most strikingly, the dominant viewBox shifts from 200×200 to 32×32, with 432 of 692 valid outputs using this icon-sized canvas. This suggests that attention-only fine-tuning learned a different subset of the training data—small icon-style SVGs

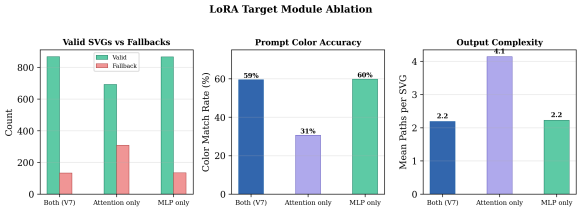

Figure 4: LoRA target module ablation across three key metrics. Left: MLP-only produces nearly as many valid SVGs as the full model (866 vs. 867), while attention-only drops to 692. Center: color match rate for attention-only (31%) is roughly half that of the other two variants (59–60%), indicating poor prompt-following. Right: attention-only generates nearly twice as many path elements per SVG (4.1 vs. 2.2), suggesting it learned a more complex but less effective output pattern.

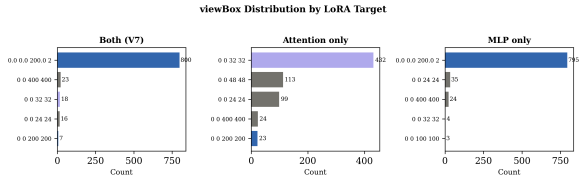

Figure 5: ViewBox distribution by LoRA target. Both (left) and MLP-only (right) overwhelmingly generate 200×200 viewBox SVGs (800 and 795 respectively), matching the training data's dominant format. Attention-only (center) shifts to 32×32 as its primary output format (432 occurrences), with secondary modes at 48×48 and 24×24—all icon-standard canvas sizes. This reveals that attention-only fine-tuning latches onto a minority subpopulation of the training data rather than learning the dominant distribution.

with grouped elements—rather than the illustration-style format that dominates the dataset.

Figure 6 shows the output length distributions. Both and MLP-only produce unimodal distributions centered around 550–650 characters, while the attention-only distribution is bimodal: one peak at 250–350 characters (the compact 32×32 icons) and another broader spread across 500–900 characters. This bimodality further confirms that attention-only fine-tuning captures a structurally different generative mode.

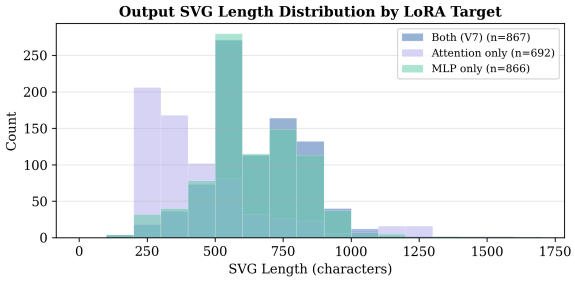

Figure 6: Output SVG length distribution by LoRA target. Both and MLP-only show similar unimodal distributions centered at 550–650 characters. Attention-only exhibits a bimodal distribution with a peak at 250–350 characters (compact 32×32 icons) and a broader secondary mode, confirming that it learned a structurally different generative pattern.

Interpretation. These results suggest that the MLP layers (feed-forward networks) are responsible for the core SVG generation capability in this task. SVG generation is primarily a pointwise numerical transformation: each output token is a coordinate value, a color hex digit, or an SVG keyword that depends more on local context than on long-range dependencies across the sequence. The MLP's role in transformers—applying learned nonlinear transformations independently at each position—is well-suited to this pattern. Attention's role in capturing cross-token relationships appears less critical for a task where the dominant output pattern is a sequence of self-contained path coordinates within a fixed structural template.

5.4 Output Quality Analysis

Analysis of V7 outputs reveals several patterns:

- **ViewBox concentration:** 80% of outputs use `viewBox="0.0 0.0 200.0 200.0"`, matching the dominant training distribution.
- **Low diversity:** Only 3 unique SVG opening structures across 867 valid outputs, indicating mode collapse under greedy decoding.
- **Color accuracy:** 64% of prompts with color words produce SVGs with matching hex codes (e.g., "orange"→`#FF9D00`).
- **Shape limitation:** The model generates correct colors but frequently produces generic shapes regardless of prompt content.

5.5 SVG Compression Analysis

Figure 7 illustrates the effect of our data compression pipeline. The dominant source of compression is float truncation (e.g., `49.884000396728516`→`49.9`), which accounts for approximately 50% of the total character reduction. Attribute removal (`filling`, `fill-opacity`) and whitespace collapsing contribute the remainder.

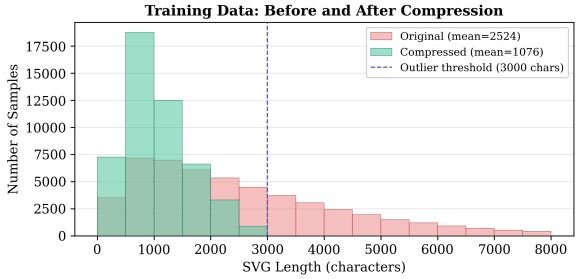

Figure 7: SVG length distribution before and after compression. Mean length drops from 2,524 to 1,076 characters (58% reduction). Outliers beyond 3,000 characters are filtered.

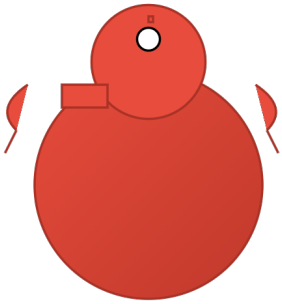

Figure 8: Given the prompt "a minimalist geometric illustration of a red cardinal bird with black accents," the base model generates a multi-element composition using circles for the body and head, paths for wings and beak, and a linear gradient for shading—a recognizable bird structure.

6 Discussion

More Valid SVGs $\neq$ Higher Score. V5 produced 906 valid SVGs but scored only 10.21, while V2 with 244 valid SVGs scored 11.83. Generic templates that pass XML validation but don't match the reference visually score worse than fewer but more diverse outputs. The 85% visual fidelity weight dominates.

Fine-Tuning as a Trade-Off. An unexpected observation is that the unfine-tuned base model (V5) occasionally produces more semantically meaningful SVGs than the fine-tuned model (V7), despite scoring lower overall. (illustrated in Figure 8). The fine-tuned model, by contrast, produces repeated rounded rectangles in the correct red color but with no semantic relationship to a bird (Figure 9). This illustrates a fundamental trade-off: fine-tuning on compressed training data collapses output diversity toward the dominant distribution mode (simple 2–3 path icons), improving average validity and score but sacrificing the base model's ability to compose novel structures for complex prompts. The competition's scoring function rewards this trade-off because the 85% visual fidelity weight means that consistently producing *something* in the right color outperforms occasionally producing a recognizable shape surrounded by many failures.

Quality vs. Reliability in DPO. The DPO result (14.19 with 197 valid SVGs vs. SFT's 14.20 with 867) reveals an important tension between output quality and generation reliability. DPO's preference optimization pushes the model toward generating SVGs that more closely match reference images, but this comes at the cost of completion rate: the model becomes less willing to produce output at all when it cannot generate a high-quality result. In a competition setting where fallbacks score low but nonzero (via color-matched generic shapes), this trade-off is approximately neutral. In a deployment setting where reliability matters, SFT would be preferred; in a setting where per-output quality matters, DPO shows clear promise if paired with a more robust decoding strategy or a retry mechanism.

Data Quality Over Model Size. The 58% SVG compression was more impactful than switching from 1.5B to 2B parameters or adjusting LoRA rank. When training data contains noise (excessive float precision), the model wastes capacity memorizing that noise rather than learning visual patterns.

MLP Dominance in SVG Generation. Our ablation shows MLP layers alone nearly replicate full model performance for SVG generation. This aligns with the nature of the task: generating SVG coordinates is primarily a point-wise numerical transformation (MLP's strength) rather than a long-range dependency task (attention's strength). The training data's dominant pattern—a few path elements with numerical coordinates in a fixed viewBox—is well-suited to MLP-based learning.

Figure 9: For the same prompt, the fine-tuned v7 model collapses to a generic template composed of repeated rounded red rectangles with small black accents. Although the output roughly matches the target color palette, it fails to capture the semantic structure of a bird—there is no identifiable body, head, beak, or wing arrangement. This example illustrates how fine-tuning on compressed SVG data can improve stylistic consistency while reducing compositional diversity and semantic fidelity.

7 Limitations

A 2B parameter model with LoRA rank 32 can learn SVG syntax and color associations but struggles with text-to-visual mapping for complex shapes. The model exhibits significant mode collapse (3 unique SVG openings) and cannot generate novel geometric constructions for arbitrary prompts.

The Unsloth–PEFT adapter key mismatch consumed significant development time and invalidated three submission versions, highlighting the fragility of mixed-framework pipelines. We lack access to reference SVGs for proxy evaluation during development, making optimization for SSIM/Edge-F1 impossible without submission-based feedback.

8 Conclusion

We demonstrate that QLoRA fine-tuning of a 2B parameter model can generate syntactically valid SVGs from text descriptions, achieving a score of 14.35. The most impactful improvements came from engineering discoveries (adapter key fix, data compression) rather than hyperparameter tuning. Our ablation on LoRA target modules reveals that MLP layers alone are sufficient for SVG generation, while attention-only fine-tuning produces a qualitatively different output distribution.

AI Tooling Disclosure

Claude (Anthropic): Coding assistance, debugging.

Acknowledgments

We thank our classmates for helpful discussions on Ed.

References

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. *Preprint*, arXiv:2305.14314.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *Preprint*, arXiv:2106.09685.

Tzu-Mao Li, Michal Lukáč, Michaël Gharbi, and Jonathan Ragan-Kelley. 2020. Differentiable vector graphics rasterization for editing and learning. *ACM Trans. Graph.*, 39(6).

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2024. Direct preference optimization: Your language model is secretly a reward model. *Preprint*, arXiv:2305.18290.

Juan A. Rodriguez, Abhay Puri, Shubham Agarwal, Issam H. Laradji, Pau Rodriguez, Sai Rajeswar, David Vazquez, Christopher Pal, and Marco Pedersoli. 2025. Starvector: Generating scalable vector graphics code from images and text. *Preprint*, arXiv:2312.11556.

A Hyperparameter Details

V1–V2 (Scores: 11.23, 11.83). Model: Qwen2.5-Coder-1.5B-Instruct. LoRA: r=32, α=64. Training: lr=2e-4, 2 epochs, batch=12. Data: 48,902 train / 998 eval. Time: 456 min on A100-40GB. Final loss: 0.290 (V1), 0.309 (V2).

V7 (Score: 14.20). Model: Qwen3.5-2B via Unsloth, 4-bit NF4. LoRA: r=32, α=64, dropout=0.0, all 7 projections. Training: lr=1e-5, cosine schedule, 3 epochs, batch=32, warmup=5%. Data: 48,375 train / 987 eval. Time: 950.7 min on RTX PRO 6000 (95 GB). Final loss: 0.4441.

V8 (Score: 14.35). Same as V7 except: lr=5e-4, 2 epochs, warmup=10%, completion-only loss masking via custom `CompletionOnlyCollator`. Batch: 8×4 gradient accumulation = 32 effective.

DPO (Score: 14.19). Two-stage pipeline on Qwen3.5-2B. Stage 1 (SFT): same as V7 configuration. Stage 2 (DPO mining): hybrid strategy with ground-truth anchoring, $k = 3$ candidates per prompt, temperature $T = 0.8$, proxy scoring via SSIM + Edge-F1 + compactness, $\sim$400 mined pairs. Stage 3 (DPO training): $\beta = 0.1$, lr=10^{-5}, 1 epoch, batch=$2 \times 4 = 8$ effective, paged AdamW 8-bit. Result: 197 valid SVGs, 803 fallbacks. Per-output quality higher than SFT but low completion rate.

B Code Repository

All code is available at: [TODO: GitHub URL]

The repository contains the complete iterative development history, organized as follows:

- `training_v1_v2ipynb` — Qwen2.5-Coder-1.5B baseline (V1–V2)
- `training_v7.ipynb` — Best SFT training + same-session inference
- `training_v8.ipynb` — Completion-only SFT variant
- `training_dpo.ipynb` — SFT + DPO pipeline
- `fix_adapter_keys.py` — Adapter key remapping utility
- `visualizations.ipynb` — All report figure generation

Earlier versions (V3–V5) are included for completeness and to document the debugging process that led to the adapter key mismatch discovery. Each notebook is self-contained and includes setup instructions via Google Drive.